



FACULTAD DE CIENCIAS **EXACTAS, NATURALES Y** **AGRIMENSURA**

Lic. En sistemas de información

Paradigmas y lenguajes

TRABAJO PRACTICO INTEGRADOR

Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Integrante

- Gonzalez Canario Nahara

Docentes

- Rodríguez, Leandro
- Mascazzini Matías

Año: 2026

Introducción

En el presente Trabajo Práctico Integrador se aborda el desarrollo de un Sistema de Semáforos Inteligentes utilizando Common Lisp como lenguaje principal de implementación. El proyecto tiene como objetivo modelar un problema real de control de tránsito urbano aplicando estrictamente los principios de la programación funcional, evitando el uso de variables mutables, estructuras iterativas imperativas y operaciones destructivas sobre los datos.

El desarrollo se organiza en tres fases complementarias. En la primera se implementa la lógica principal del sistema semafórico mediante funciones puras, recursividad y funciones de orden superior. En la segunda fase se investiga e integra una librería externa del ecosistema Lisp (`local.time`), ampliando las capacidades del sistema mediante herramientas modernas de gestión de dependencias. Finalmente, en la tercera fase se realiza un comparativo entre Common Lisp y otro lenguaje de programación asignado (`elixir`), analizando similitudes, diferencias y características propias de cada paradigma y entorno de ejecución.

Además de la implementación técnica, este trabajo busca fomentar la autonomía en el aprendizaje de nuevas tecnologías, el desarrollo del pensamiento algorítmico y la capacidad de análisis crítico sobre las decisiones de diseño adoptadas. Para ello, se documentan los problemas encontrados durante el proceso de desarrollo y las estrategias utilizadas para resolverlos.

Objetivos del Trabajo Práctico

- **Modelar un problema del mundo real:** Traducir los requisitos funcionales de un sistema de tráfico urbano a código LISP utilizando el paradigma funcional.
- **Desarrollar autonomía y pensamiento algorítmico:** Resolver la integración de herramientas externas y el aprendizaje de una tecnología nueva con el mínimo de asistencia docente.
- **Análisis Crítico y Metacognición:** Evaluar y tipificar las funciones creadas, comprendiendo sus implicancias en la memoria y el flujo de datos, y comparar cómo diferentes lenguajes abordan un mismo problema lógico.

Contexto del Problema

Las ciudades modernas requieren sistemas de tráfico inteligentes para optimizar el flujo vehicular y garantizar la seguridad vial. Su equipo ha sido contratado para desarrollar el núcleo lógico de un sistema embebido que controlará semáforos en intersecciones críticas de la ciudad. El mismo fue implementado en Common Lisp.

Desarrollo del trabajo

Etapa 1: análisis del problema, para representar el comportamiento del semáforo se diseñó un modelo basado en una máquina de estados finitos compuesta por tres estados principales: **Rojo (R)**, **Amarillo (Am)** y **Verde (V)** Cuando el semáforo se encuentra en estado Rojo, al finalizar el tiempo asignado cambia al estado Amarillo. Posteriormente, el estado Amarillo realiza la transición hacia Verde. Finalmente, el estado Verde retorna al estado Rojo, completando así el ciclo del semáforo.

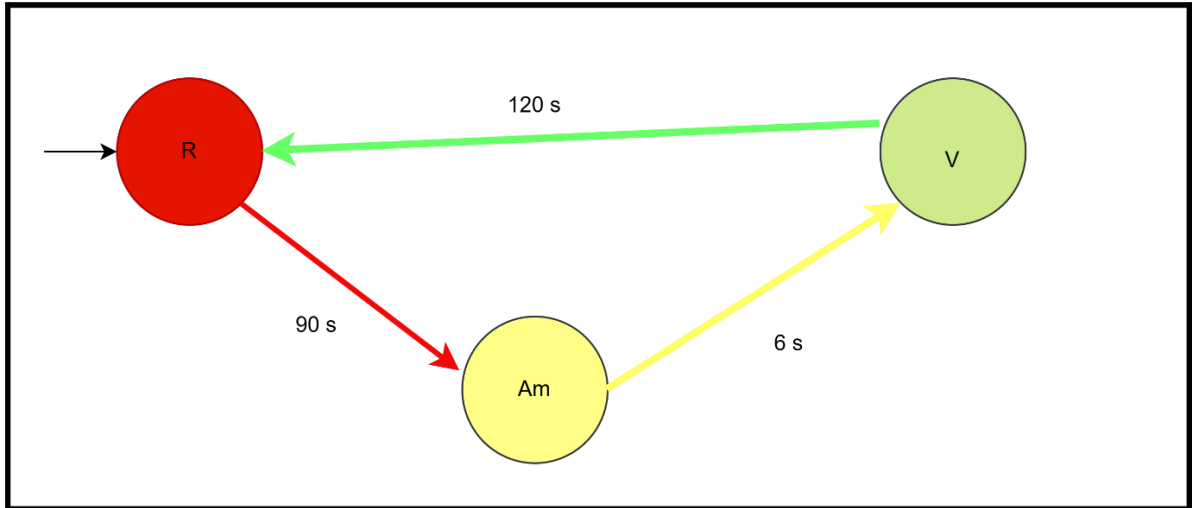


imagen1. Representación de un ciclo de semáforo

El diagrama de estados muestra las relaciones entre cada color y el orden en que ocurren las transiciones, suponiendo que las transiciones no consumen tiempo.

Transiciones validas

(transición 'en-rojo 'amarillo) → ('en-rojo "cambiar-a-amarillo")

(transición 'en-amarillo 'verde) → ('en-amarillo "cambiar-a-verde") (transición

'en-verde rojo) → ('en-verde "cambiar-a-rojo")

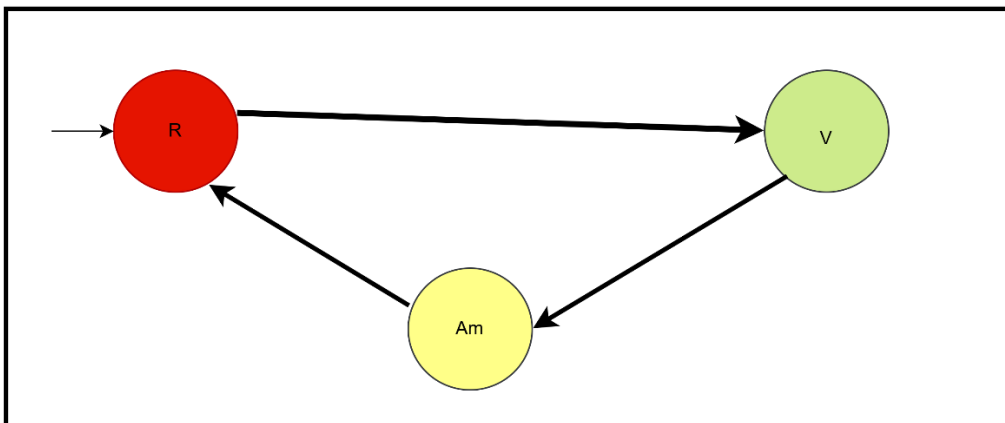


Imagen2. Transiciones invalidas

Transiciones invalidas

(transición 'en-rojo 'verde) ('en-rojo "acción-por-defecto")

(transición 'en-verde 'amarillo) → ('en-verde "acción-por-defecto")

(transición 'en-amarillo 'rojo) → ('en-amarillo "acción-por-defecto")

Requerimiento 1: estados de transición

```
(defun transicion (color-actual cambiar-a)
```

```
  (cond ((and (eq color-actual 'en-rojo) (eq cambiar-a 'amarillo))
        (list 'en-rojo "cambiar-a-amarillo"))
        ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'verde))
        (list 'en-amarillo "cambiar-a-verde"))
        ((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo))
        (list 'en-verde "cambiar-a-rojo"))
        (t
         (list color-actual "accion-por-defecto")))))
```

La función transicion fue diseñada para modelar el comportamiento básico del semáforo mediante una serie de transiciones válidas entre estados. Para ello se utilizó la estructura cond, que permite evaluar distintas condiciones y ejecutar la acción correspondiente según el estado actual y el color de destino. Si la combinación recibida no corresponde a una transición válida, la función retorna el estado actual junto con la acción por defecto mediante la cláusula t del cond.

```
(defun control-cambio-color (color-actual cambiar-a)
  (cond ((or (not (eq color-actual 'en-rojo)) (not (eq color-actual 'en-amarillo)) (not (eq color-actual 'en-verde)))) NIL)
        ((or (not (eq cambiar-a 'rojo)) (not (eq cambiar-a 'amarillo)) (not (eq cambiar-a 'verde)))) NIL)
        ((or (listp color-actual) (listp color-actual)) NIL)
        (T (transicion color-actual cambiar-a)))
  ))
```

```
(transicion 'en-morado 'rojo)
```

```
(control-cambio-color 'en-morado 'amarillo)
```

```
(control-cambio-color 'en-rojo 'rosado)
```

(control-cambio-color '(1 2) '(3 4))

(transicion '(1 2) '(3 4))

La función control-cambio-color fue creada con el objetivo de garantizar que los datos ingresados correspondan a estados y colores válidos del sistema de semáforos, evitando que se procesen valores inexistentes o estructuras de datos incorrectas.

La necesidad de esta función surgió al observar que la función transicion podía recibir parámetros no contemplados en el modelo del problema, por ejemplo

(control-cambio-color 'en-morado 'amarillo) o (control-cambio-color 'en-rojo 'rosado)

Dado que los colores en-morado y rosado no forman parte de los estados definidos para el semáforo, además la función transicion devuelve una acción por defecto cuando recibe una combinación no válida. Sin embargo, se consideró que devolver una acción por defecto podía interpretarse como si el sistema hubiera ejecutado algún tipo de transición. Por este motivo se incorporó una validación previa que retorna NIL, indicando explícitamente que la operación fue rechazada y que no se realizó ninguna acción sobre el semáforo.

Además, se incorporó una validación para evitar que se envíen listas como argumentos, ya que el sistema fue diseñado para trabajar únicamente con átomos que representen estados y colores.

	A	B	C	D
1	color-actual	cambiar-a	Resultado Esperado	Resultado Obtenido
2	en-rojo	amarillo	Aparezca la lista siguiente: (en-rojo "cambiar-a-amarillo")	OK
3	en-amarillo	verde	Aparezca la lista siguiente: (en-amarillo "cambiar-a-verde")	
4	en-verde	rojo	Aparezca la lista siguiente: (en-verde "cambiar-a-rojo")	
5	en-rojo	verde	Aparezca la lista siguiente: (en-rojo "accion-por-defecto")	
6	en-verde	amarillo	Aparezca la lista siguiente: (en-rojo "accion-por-defecto")	
7	en-amarillo	rojo	Aparezca la lista siguiente: (en-rojo "accion-por-defecto")	
8	en-morado	verde	NIL	(EN-MORADO "accion-por-defecto")
9	en-rojo	rosado	NIL	(EN-ROJO "accion-por-defecto")
10	(1 2)	(3 4)	NIL	((1 2) "accion-por-defecto")
11	en-amarillo	(1 2)	NIL	
12	(1 2)	verde	NIL	
13	(1 2)	rosado	NIL	
14	en-morado	(1 2)	NIL	

Imagen3. Escenarios de pruebas

Requerimiento 2: temporizador automático

Antes de implementar la función timer, se realizó un esquema visual para representar cómo se distribuyen los tiempos de cada color dentro del ciclo completo del semáforo.

La idea consistió en visualizar el ciclo como una secuencia continua de estados donde cada color permanece activo durante una cantidad determinada de segundos

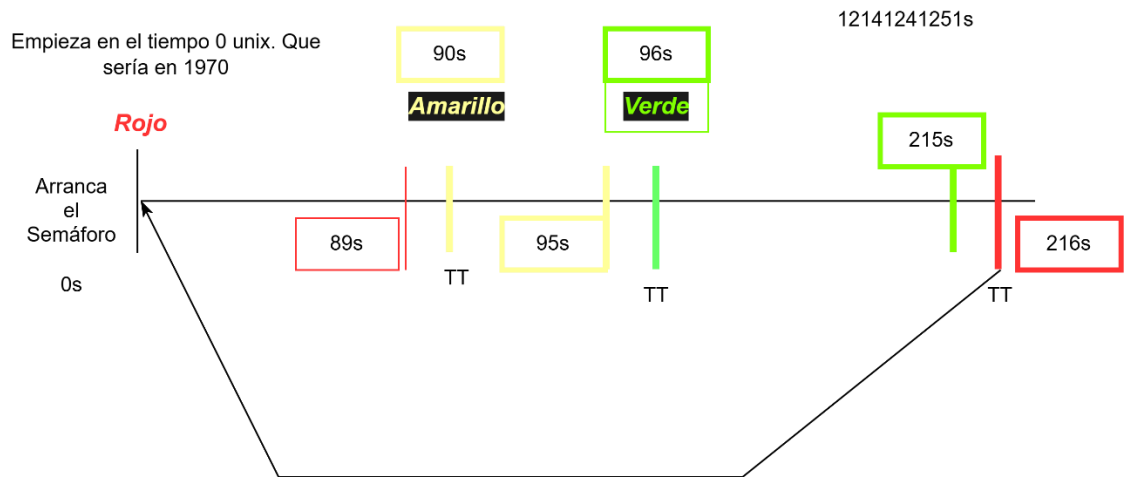


Imagen4. Intervalos de tiempo del funcionamiento del semáforo

La solución se basó en considerar que el semáforo funciona de manera cíclica. Por este motivo se utilizó la operación módulo (mod) con el valor total del ciclo (216 segundos), obteniendo así la posición exacta del instante analizado dentro del ciclo actual.

A partir de ese valor se definieron tres rangos:

- Desde el segundo 0 hasta el 89: estado en-rojo.
- Desde el segundo 90 hasta el 95: estado en-amarillo.
- Desde el segundo 96 hasta el 215: estado en-verde.

La implementación inicial fue la siguiente:

```
(defun timer_transition(data_time)

  (cond
    ((< (mod data_time 216) 90) 'en-rojo)
    ((and (>= (mod data_time 216) 90)
          (<= (mod data_time 216) 95))
     'en-amarillo)
    ((and (>= (mod data_time 216) 96)
          (<= (mod data_time 216) 215))
     'en-verde)))
```

En esta versión se utilizó un ciclo fijo de 216 segundos, resultado de sumar las duraciones establecidas para cada color: 90 segundos (Rojo)

+ 6 segundos (Amarillo)

+ 120 segundos (Verde)

216 segundos

La función emplea la operación módulo (mod) para determinar la posición exacta del tiempo recibido dentro del ciclo completo. Dependiendo del intervalo en el que se encuentre dicho valor, retorna el estado correspondiente del semáforo.

Si bien esta solución cumplía con los requisitos iniciales, presentaba una limitación importante: los tiempos estaban escritos directamente en el código.

Con el objetivo de lograr una implementación más flexible, se desarrolló una segunda versión denominada `timer_2`.

```
(defun timer_2 (tiempo-unix rojo amarillo verde)
  (cond
    ((and (>= (mod tiempo-unix (+ rojo amarillo verde)) 0)
          (<= (mod tiempo-unix (+ rojo amarillo verde))
              (- rojo 1))))
    'en-rojo)

    ((and (>= (mod tiempo-unix (+ rojo amarillo verde))
              rojo)
          (< (mod tiempo-unix (+ rojo amarillo verde))
              (+ rojo amarillo))))
    'en-amarillo)

    (T 'en-verde)))
```

La principal diferencia es que los tiempos de duración ya no se encuentran codificados dentro de la función. En su lugar, son recibidos como parámetros.

Esto permite reutilizar la misma lógica para cualquier configuración de semáforo sin necesidad de modificar el código fuente.

Incorporación de Configuración Externa mediante JSON se incorporó la biblioteca `cl-json` utilizando `Quicklisp`.

```
(defun get_tiempo_colores ()
  (with-open-file
    (stream "config.json"
      :direction :input
      :if-does-not-exist nil)

    (unless stream
      (error "No se pudo abrir el archivo"))

    (let ((datos (json:decode-json stream)))
      (list
        (cdr (assoc :ROJO datos))
        (cdr (assoc :AMARILLO datos))
        (cdr (assoc :VERDE datos))))))
```

Esta función abre el archivo JSON, lee su contenido y devuelve una lista con los tiempos correspondientes a cada color.

Por ejemplo:

```
{
  "ROJO": 90,
  "AMARILLO": 6,
  "VERDE": 120
}
```

La ventaja de esta solución es que cualquier cambio en la temporización puede realizarse modificando únicamente el archivo de configuración, sin necesidad de alterar el código del programa.

Durante las pruebas se observó que podían ingresarse valores inválidos, como tiempos negativos o duraciones iguales a cero.

Para evitar estos problemas se desarrolló la función `control_timer`.

```
(defun control_timer(tiempo-unix rojo amarillo verde)
  (cond
    ((< tiempo-unix 0)
     "El tiempo no puede ser menor a cero")

    ((<= rojo 0)
     "El tiempo del semáforo en rojo no puede ser cero o menos")

    (T
     (timer_2 tiempo-unix rojo amarillo verde))))
```

Esta función actúa como una capa de validación previa a la ejecución del temporizador. Si detecta datos inconsistentes, informa el error correspondiente. En caso contrario, pasa el procesamiento a la función `timer_2`.

Requerimiento 3: sistema de auditoria

El objetivo de este requerimiento fue desarrollar un mecanismo de auditoría que permitiera conocer qué transición realizó el semáforo y en qué momento ocurrió.

La información registrada debía seguir el formato:

Tiempo <fecha-hora>: la luz ha cambiado de <color-anterior> a <color-nuevo>

Inicialmente los tiempos se encontraban en formato Universal Time, lo que dificultaba la lectura de los registros.

Para solucionar este problema se implementó la función:

```
(defun universal-to-datestring (utime &optional (timezone 0))

  (multiple-value-bind (seg min hora dia mes anio)
    (decode-universal-time utime timezone)

    (format nil "~4d~2,'0d~2,'0d~2,'0d~2,'0d~2,'0d"
            anio mes dia hora min seg)))
```

La función utiliza `decode-universal-time` para separar el tiempo en sus componentes (año, mes, día, hora, minuto y segundo) y posteriormente emplea `format` para construir una cadena legible. Por ejemplo:

`(universal-to-datestring (get-universal-time))` puede generar una salida similar a:
2025-06-13 10:30:00

Esta conversión permitió que los registros fueran comprensibles para una persona sin necesidad de interpretar timestamps numéricos.

Para identificar los instantes exactos en que ocurre una transición se desarrolló la función:

```
(defun traffic_ligth_status(data_time)
```

```
(cond
```

```
((= (mod data_time 216) 90)
```

```
"La luz ha cambiado de rojo a amarillo")
```

```
((= (mod data_time 216) 96)
```

```
"La luz ha cambiado de amarillo a verde")
```

```
((= (mod data_time 216) 0)
```

```
"La luz ha cambiado de verde a rojo")
```

```
(T
```

```
(timer_transition data_time))))
```

La idea surgió a partir del análisis realizado durante el desarrollo del temporizador.

Sabíamos que:

- El rojo finaliza en el segundo 90.
- El amarillo finaliza en el segundo 96.
- El ciclo completo termina en el segundo 216.

Por lo tanto, si el resultado de: (mod data_time 216) coincide con alguno de esos valores, significa que el semáforo acaba de cambiar de estado. En lugar de devolver únicamente el color actual, la función devuelve un mensaje descriptivo indicando exactamente qué transición ocurrió.

Cuando se incorporó la carga de tiempos desde archivos JSON, fue necesario generalizar la lógica anterior.

Para ello se desarrolló:

```
(defun traffic_ligth_status_from_config  
  (data_time rojo amarillo verde)
```

```
(cond
```

```
((= (mod data_time (+ rojo amarillo verde))
```

```
rojo)
```

```
"La luz ha cambiado de rojo a amarillo")
```

```
((= (mod data_time (+ rojo amarillo verde))
```

```
(+ rojo amarillo))
```

```
"La luz ha cambiado de amarillo a verde")
```

```
((= (mod data_time (+ rojo amarillo verde))
```

```
0)
```

```
"La luz ha cambiado de verde a rojo")
```

```
(T
```

```
(timer_2 data_time rojo amarillo verde))))
```

A diferencia de la versión anterior, los límites de cada transición ya no se encuentran fijos en el código, sino que se calculan dinámicamente utilizando los tiempos configurados para cada color.

De esta manera el sistema continúa funcionando correctamente aunque se modifiquen las duraciones del semáforo.

Una vez detectadas las transiciones, se implementó la función encargada de registrar los eventos:

```
(defun registrar_cambios(tiempo-inicial)
```

```
(format t
  "Tiempo ~a -> ~a~%"
  (universal-to-datestring tiempo-inicial 0)
  (traffic_ligth_status tiempo-inicial)))
```

Esta función imprime en pantalla la fecha y hora junto con el estado correspondiente del semáforo.

Por ejemplo:

Tiempo 2025-06-13 10:30:00 -> La luz ha cambiado de rojo a amarillo

Durante las pruebas se observó que registrar todos los estados generaba una gran cantidad de información innecesaria.

Por ejemplo:

en-rojo en-rojo

en-rojo en-rojo

en-rojo

Esto dificultaba localizar las transiciones importantes.

Para resolver este inconveniente se desarrolló una segunda versión:

```
(defun registrar_cambios_2 (tiempo-inicial)
```

```
(let ((estado
      (traffic_ligth_status tiempo-inicial)))
```

```
(unless
 (member estado
          '(en-rojo en-
            amarillo
              en-verde)))
```

```
(format t
  "Tiempo ~a -> ~a~%"
  (universal-to-datestring tiempo-inicial 0)
  estado))))
```

La función utiliza unless para ignorar los estados normales y registrar únicamente los momentos en que realmente ocurre un cambio de color.

Como parte del requerimiento 2 se incorporó la biblioteca local-time mediante Quicklisp.

Para ello se adaptó la función parse-fecha:

```
(local-time:encode-timestamp
```

```
0 second minute hour day month year  
:offset 0)
```

Esta implementación permitió trabajar con fechas y horarios de manera más robusta que utilizando únicamente funciones básicas de Common Lisp.

Posteriormente se utilizó:

```
(local-time:timestamp-difference  
(local-time:now)  
(parse-fecha fecha-hora-str))
```

para calcular diferencias temporales entre una fecha ingresada por el usuario y el momento actual.

Requerimiento 4: Análisis de Ciclos Semafóricos

Función duracion-ciclo

La función recibe una lista que contiene los tiempos de duración de cada estado del semáforo y calcula la duración total del ciclo semafórico mediante la suma de todos sus elementos.

```
(defun duracion-ciclo (lista-tiempos)  
  (reduce #'+ lista-tiempos)  
)
```

Se utiliza la función de orden superior reduce, que aplica sucesivamente la operación de suma (+) a todos los elementos de la lista hasta obtener un único resultado.

Por ejemplo

```
(duracion-ciclo '(30 5 25))
```

realiza internamente:

```
(+ 30 5 25)
```

obteniendo:

```
60
```

que representa la duración total del ciclo semafórico.

Función recomendacion-ciclo

Esta función analiza la duración total calculada para un ciclo semafórico y genera una recomendación basada en los estándares de ingeniería de tránsito indicados en la consigna.

```
(defun recomendacion-ciclo (tiempo-analizado)
  (cond
    ((< tiempo-analizado 35)
      "Tiempo muy bajo. Se recomienda revisar el tiempo total")

    ((> tiempo-analizado 150)
      "Tiempo muy alto. Se recomienda revisar el tiempo total")

    (t
      "Tiempo dentro del rango [35; 150]")
  )
)
```

La estructura cond evalúa distintas condiciones:

- Si la duración es menor a 35 segundos, se considera un ciclo demasiado corto.
- Si la duración supera los 150 segundos, se considera demasiado largo.
- En cualquier otro caso, se encuentra dentro del rango recomendado.

Esta validación permite verificar que la configuración del semáforo sea adecuada para la percepción y comportamiento de los conductores.

Requerimiento 5: Planificación Temporal

Antes de implementar la función, se observó que cada ciclo está formado por la secuencia rojo → amarillo → verde, por lo que su duración total se obtiene sumando los tiempos de cada color. Con los valores establecidos en el sistema, un ciclo dura 216 segundos ($90 + 6 + 120$). A partir de esta idea, se concluyó que para conocer cuántos ciclos ocurren en un período determinado solo es necesario convertir el tiempo a segundos y dividirlo por la duración total de un ciclo. Este razonamiento permitió diseñar la función `ciclos-por-tiempo`, que calcula automáticamente la cantidad de ciclos completos dentro del intervalo analizado.

Supongamos que se empieza en
el inicio del ciclo

15 minutos

¿cuántos ciclos rojo->amarillo->verde->rojo hay?

Ciclo Completo=TR+TA+TV (suponiendo que las transiciones tienen tiempo despreciable)

$$15 \cdot 60 = 900s$$

$$900 s = X \cdot (TR + TA + TV)$$

$$\text{Ciclo Completo} = 90 + 6 + 120 = 216s$$

Si vos queres analizar en 216 s cuántos ciclos hay ¿qué resultado te da? 1 ciclo

Si vos queres analizar en 432 s ¿cuántos ciclos hay? 2 ciclos

$$216 = X \cdot 216 \Rightarrow x = 216 / 216 = 1$$

$$432 = X \cdot 216 \Rightarrow x = 432 / 216 = 2$$

$$\text{Nro De Ciclos} = \text{Tiempo Analizado} / \text{Tiempo Total Del Ciclo}$$

Imagen5. calcular la cantidad de ciclos dentro de un intervalo de 15min

```
(defun ciclos-por-tiempo (minutos)
  (floor (* 60 minutos) (reduce #'(get_tiempo_colores) ))
)
```

(ciclos-por-tiempo 20)

la función ciclos-por-tiempo, que recibe como parámetro una duración expresada en minutos. Realiza dos operaciones principales:

1. Convierte los minutos ingresados a segundos:
(`* 60 minutos`)
2. Calcula la duración total de un ciclo del semáforo sumando los tiempos configurados para los colores rojo, amarillo y verde:
(`reduce #'(get_tiempo_colores)`)

Finalmente, mediante la función floor, se obtiene la cantidad de ciclos completos que pueden ejecutarse dentro del tiempo especificado.

(`floor tiempo-total-segundos duracion-ciclo`)

La utilización de floor garantiza que únicamente se contabilicen ciclos completos, descartando cualquier fracción de ciclo que pudiera quedar incompleta.

Ejemplo de ejecución

(ciclos-por-tiempo 20)

Si los tiempos configurados son:

- Rojo: 90 segundos
- Amarillo: 6 segundos
- Verde: 120 segundos

La duración total de un ciclo es:

$$90 + 6 + 120 = 216 \text{ segundos}$$

Para 20 minutos: $20 \times 60 =$
1200 segundos Por lo tanto:
 $1200 \div 216 = 5,55$
Aplicando floor, el resultado obtenido es:
5 ciclos completos

Requerimiento 6: Informe de Distribución Temporal

La función principal utilizada para este análisis fue `distribucion_colores`.

```
(defun distribucion_colores  
  (tiempo_inicial tiempo_analizado tiempo_max  
    cr ca cv tr ta tv)
```

La idea fue recorrer segundo por segundo un intervalo de tiempo determinado y contabilizar cuántas veces el semáforo se encontraba en rojo, amarillo o verde. Los parámetros: `cr`, `ca`, `cv` funcionan como acumuladores que almacenan la cantidad de veces que aparece cada color durante el recorrido.

Se eligió implementar la solución mediante recursividad de cola. La condición de corte es:

```
((= tiempo_analizado tiempo_max)  
 (list cr ca cv))
```

Cuando se alcanza el tiempo máximo establecido, la función finaliza y devuelve una lista con las frecuencias acumuladas.

Por ejemplo: (`frecuencia_rojos`
`frecuencia_amarillos`
`frecuencia_verdes`)

Durante cada llamada recursiva se consulta el estado actual del semáforo utilizando la función `timer_2`.

Para el color rojo:

```
((eq (timer_2  
      (+ tiempo_inicial tiempo_analizado)  
      tr ta tv) 'en-rojo)
```

```
(distribucion_colores  
 tiempo_inicial (+  
 tiempo_analizado 1)  
 tiempo_max  
 (+ cr 1)  
 ca cv  
 tr ta tv))
```

Si el temporizador devuelve `en-rojo`, se incrementa únicamente el contador de rojos. De forma similar se realiza el conteo para amarillo:

```
((eq (timer_2
```

```
(+ tiempo_inicial tiempo_analizado)
tr ta tv)
'en-amarillo)
```

```
(distribucion_colores
tiempo_inicial (+
tiempo_analizado 1)
tiempo_max
cr
(+ ca 1)
cv tr ta
tv))
```

Y finalmente cualquier otro caso corresponde al color verde:

```
(T
```

```
(distribucion_colores
tiempo_inicial (+
tiempo_analizado 1)
tiempo_max
cr ca
(+ cv 1)
tr ta tv))
```

La función fue pensada como una simulación temporal.

Por ejemplo:

Tiempo inicial = 100000

```
100000 + 0
100000 + 1
100000 + 2
100000 + 3
...
100000 + 3600
```

En cada segundo se consulta el estado del semáforo y se actualiza el contador correspondiente. De esta manera se obtiene una visión completa de cómo se distribuyen los colores dentro del intervalo analizado.

Una vez obtenidas las frecuencias, se desarrolló una segunda función encargada de calcular los porcentajes de aparición de cada color.

```
(defun calcular_porcentajes (lista)
```

```
(list
(* (/ (nth 0 lista)
(reduce #'+ lista))
100.0)
```

```
(* (/ (nth 1 lista)
(reduce #'+ lista))
100.0)
```

```
(* (/ (nth 2 lista)
      (reduce #'(lambda (x y)
                  (+ x y))
              lista))
      100.0)))
```

La función recibe una lista con las frecuencias obtenidas previamente.

Por ejemplo: (100 10 200) donde:

- 100 representa la cantidad de veces que apareció el rojo.
- 10 representa la cantidad de veces que apareció el amarillo.
- 200 representa la cantidad de veces que apareció el verde.

Uso de Funciones de Orden Superior

Para obtener el total de observaciones se utilizó la función de orden superior:

```
(reduce #'(lambda (x y)
            (+ x y))
        lista)
```

Esta función suma todos los elementos de la lista y permite calcular el total de muestras analizadas.

Posteriormente cada frecuencia se divide por ese total y se multiplica por 100 para obtener su porcentaje.

La fórmula aplicada es:

Porcentaje = (Frecuencia / Total) × 100

Ejemplo de Ejecución

La obtención de porcentajes se realiza mediante:

```
(calcular_porcentajes
 (distribucion_colores
  (get-unix-time)
  0
  3600
  0 0 0
  (car (get_tiempo_colores))
  (car (cdr (get_tiempo_colores)))
  (car (cddr (get_tiempo_colores))))))
```

En este ejemplo se analiza una hora completa (3600 segundos) a partir del instante actual.

Primero se cuentan las frecuencias de cada color y luego se calculan los porcentajes correspondientes.

Requerimiento 7: Aseguramiento de la calidad

Casos de prueba propuestos para el Requerimiento 7 (Aseguramiento de la Calidad).

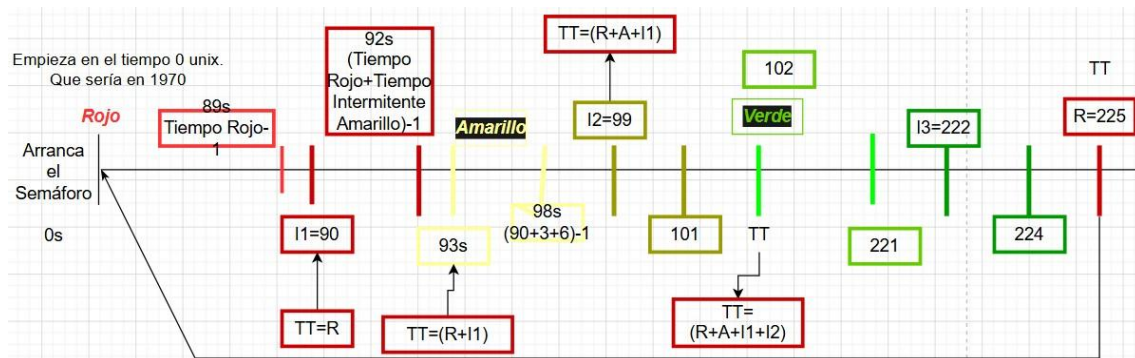
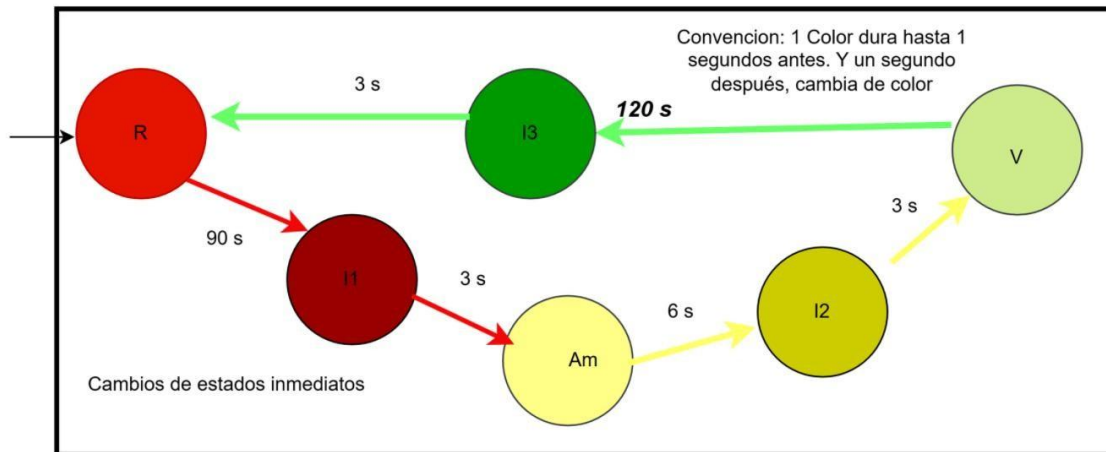
Requerimiento	Función	Entrada	Resultado Esperado	Observación
Req.1	transicion	(transicion 'enrojo 'amarillo)	(en-rojo "cambiar-a-amarillo")	Transición válida
Req.1	transicion	(transicion 'enrojo 'verde)	(en-rojo "accion-por-defecto")	Transición inválida

Req.1	control-cambiocolor	(control-cambio-color 'enmorado 'amarillo)	NIL	Color inexistente
Req.1	control-cambiocolor	(control-cambio-color '(1 2) '(3 4))	NIL	Listas no permitidas
Req.2	timer_2	(timer_2 50 90 6 120)	en-rojo	Dentro del intervalo rojo
Req.2	timer_2	(timer_2 92 90 6 120)	en-amarillo	Dentro del intervalo amarillo
Req.2	timer_2	(timer_2 150 90 6 120)	en-verde	Dentro del intervalo verde
Req.2	control_timer	(control_timer -1 90 6 120)	Mensaje de error	Tiempo negativo
Req.2	control_timer	(control_timer 100 0 6 120)	Mensaje de error	Rojo inválido
Req.3	universal-todatestring	(universal-todatestring tiempo)	YYYY-MM-DD HH:MM:SS	Conversión correcta
Req.3	traffic_ligth_status	mod=90	La luz ha cambiado de rojo a amarillo	Detecta transición
Req.3	registrar_cambios_2	Estado normal	No registra	Filtra estados repetidos
Req.4	ciclos-portiempo	(ciclos-portiempo 20)	5	$1200/216=5$ ciclos completos
Req.5	ciclos-portiempo	(ciclos-portiempo 60)	16	Planificación en 1 hora
Req.6	dis-tribucion_colores	3600 segundos	(rojos amarillos verdes)	Devuelve frecuencias
Req.6	calcular_porcentajes	(100 10 200)	(32.26 3.23 64.52 aprox.)	Calcula porcentajes

Iteración 2:

Extensión 1: Intermittencia de Seguridad

El modelo del semáforo se modifica de la siguiente manera



Usando las imágenes anteriores, realicé las siguientes funciones para contemplar su comportamiento

```
(defun get_tiempo_colores ())
```

```
(with-open-file (stream
"c:/Users/espino/OneDrive/Documentos/LISI/Paradigma/json/config.json.txt"
```

```
:direction :input
```

```
:if-does-not-exist nil)
```

```

(unless stream
  (error "No se pudo abrir el archivo"))

(let ((datos (json:decode-json stream))) (list (cdr (assoc :ROJO datos)) (cdr (assoc :AMARILLO
datos)) (cdr (assoc :VERDE datos))

  (cdr (assoc :itr datos)) (cdr (assoc :ita datos)) (cdr (assoc :itv datos))) )))

```

Esta función permite obtener los colores para las intermitencias. El archivo config.json pasará a tener el siguiente formato

```

{
  "rojo": 70,
  "amarillo":6,
  "verde":120,
  "itr":3,
  "ita":3,
  "itv":3
}

```

Donde itr, ita, itv son los tiempos de intermitencia para el rojo, amarillo y verde respectivamente.

```

(defun intervalo_rojo (tiempo-unix total rojo iter_r)

```

```

  (cond ((and (>= (mod tiempo-unix total) 0) (<= (mod tiempo-unix total) (- rojo 1)) T)
        (T NIL))))

```

La función intervalo_rojo permite saber si el semáforo se encuentra en el color rojo para el tiempo pasado como dato. De ser así, retornará T y en caso contrario retornará NIL.

Un razonamiento similar se aplica para las siguientes funciones

```

(defun intervalo_iter_rojo (tiempo-unix total rojo iter_r)

(cond

((and (>= (mod tiempo-unix total) rojo) (< (mod tiempo-unix total) (+ rojo iter_r))) T) (T
NIL)

)

```

)

```
(defun intervalo_amarillo (tiempo-unix total rojo iter_r amarillo)
```

```
  (cond
```

```
    ((and (>= (mod tiempo-unix total) (+ rojo iter_r)) (< (mod tiempo-unix total) (+ rojo iter_r
    amarillo)))) T)
```

```
    (T NIL))
```

)

```
(defun intervalo_iter_amarillo(tiempo-unix total rojo iter_r amarillo iter_a)
```

```
(cond
```

```
  ((and (>= (mod tiempo-unix total) (+ rojo iter_r amarillo)) (< (mod tiempo-unix total) (+
  rojo iter_r amarillo iter_a))) T)
```

```
  (T NIL))
```

)

```
(defun intervalo_verde(tiempo-unix total rojo iter_r amarillo iter_a verde)
```

```
  (cond
```

```
    ((and (>= (mod tiempo-unix total) (+ rojo iter_r amarillo iter_a)) (< (mod tiempo-unix
    total) (+ rojo iter_r amarillo iter_a verde)))) T)
```

```
    (T NIL))
```

)

Con esas funciones, creo la función timer_iter

```
(defun timer_iter (tiempo-unix rojo amarillo verde iter_r iter_a iter_v)
```

```
  (cond
```

```
    ((intervalo_rojo tiempo-unix (+ rojo amarillo verde iter_r iter_a iter_v) rojo iter_r) 'en-rojo)
```

```
    ((intervalo_iter_rojo tiempo-unix (+ rojo amarillo verde iter_r iter_a iter_v) rojo iter_r) 'en-
    intermitente-rojo)
```

```

        ((intervalo_amarillo tiempo-unix (+ rojo amarillo verde iter_r iter_a iter_v) rojo iter_r
        amarillo) 'en-amarillo)

        ((intervalo_iter_amarillo tiempo-unix (+ rojo amarillo verde iter_r iter_a iter_v)
        rojo iter_r amarillo iter_a) 'en-intermitente-amarillo)

        ((intervalo_verde tiempo-unix (+ rojo amarillo verde iter_r iter_a iter_v) rojo
        iter_r amarillo iter_a verde) 'en-verde)

        (T 'en-intermitente-verde)

    )

```

)

Esta función controla en cuál intervalo se encuentra el semáforo en el tiempo pasado. De esa manera, mostrará en pantalla el color correspondiente.

La siguiente función es esta

```

(defun traffic_ligth_status_from_config(data_time rojo amarillo verde iter_r iter_a iter_v)

(cond

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) rojo) "La luz ha cambiado de
  rojo a rojo-intermitente" )

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) (+ rojo iter_r)) "La luz ha
  cambiado de rojo-intermitente a amarillo" )

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) (+ rojo iter_r amarillo)) "La luz
  ha cambiado amarillo a amarillo-intermitente" )

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) (+ rojo iter_r amarillo iter_a))
  "La luz ha cambiado amarillo-intermitente a verde" )

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) (+ rojo iter_r amarillo iter_a
  verde)) "La luz ha cambiado verde a verde-intermitente" )

  ((= (mod data_time (+ rojo amarillo verde iter_r iter_a iter_v)) 0) "La luz ha cambiado
  verdeintermitente a rojo" )

  (T (timer_iter data_time rojo amarillo verde iter_r iter_a iter_v))

```

))

Esta función es una modificación de la función que ya existe con el mismo nombre. Lo que hará es mostrar qué transición se está produciendo y, en caso de no producirse una transición, mostrará en qué color se encuentra el semáforo para el tiempo pasado como parámetro.

Extensión 2: Persistencia de Datos

Para esta parte del trabajo, se implementó la siguiente función

```
(defun informe (tiempo-inicial &optional (archivo " C:/Users/Usuario/Downloads/TPIFuncional-
2026-Grupo[9]/docs/informacion_semaforo.txt"))

(let ((estado (traffic_ligth_status tiempo-inicial)))

  (format t "DEBUG: estado = ~a~%" estado) ; imprime en pantalla para depuración

  (unless (member estado '(en-rojo en-amarillo en-verde))

    (with-open-file (out archivo

      :direction :output

      :if-exists :append

      :if-does-not-exist :create)

      (format out "~%Tiempo ~a -> ~a~%"

        (universal-to-datestring tiempo-inicial 0)

        estado)

      (finish-output out))))))
```

La función informe, emplea el método traffic_ligth_status para saber en qué estado se encuentra el semáforo. Si se encuentra en los estados 'en-rojo', 'en-amarillo' o 'en-verde', el unless lo ignora. En caso contrario, almacena ese dato en el archivo indicado anteriormente.

El problema principal que surgió aquí es que era difícil generar un bucle sin usar dotimes, loop, y demás. Un primer enfoque era esta función

```
(defun guardar_informe(tiempo_inicial ruta_archivo tiempo_max)
```

```
(dotimes (i tiempo_max)

  (informe (+ (get-universal-time) i) ruta_archivo))

)
```

Que, al ejecutarla, guardaba correctamente los datos. Para la prueba, se uso un entorno denominado LispIDE, un entorno que resultó muy difícil configurar la librería quicklisp.

Al ejecutar esa función:

```
(guardar_informe (get-universal-time)
"c:/Users/epin/OneDrive/Documentos/LISI/Paradigma/registro_semaforo.txt" 3600)
```

Se pudieron almacenar correctamente los resultados

```
Tiempo 2026-06-16 23:02:24 -> La luz ha cambiado de verde a rojo
Tiempo 2026-06-16 23:03:54 -> La luz ha cambiado de rojo a amarillo
Tiempo 2026-06-16 23:04:00 -> La luz ha cambiado de amarillo a verde
```

Pero esa función no cumple los requisitos del trabajo. La segunda propuesta fue la siguiente

```
(defun guardar_informe (i max_iter)

  (when (< i max_iter)

    (informe (+ (get-universal-time) i))

    (guardar_informe (1+ i) max_iter)))
```

La cual no emplea dotime, loop y demás, sino que es una recursión de cola. Pero, al llamarla en LispIDE nos arroja el siguiente resultado

```
DEBUG: estado = EN-VERDE
DEBUG: estado = EN-VERDE
DEBUG: estado = EN-VERDE
DEBUG: estado = EN-VERDE
DEBUG: estado = EN-VERDE
DEBUG: estado =
*** - Program stack overflow. RESET
```

Indicando un desbordamiento de pila. Para intentar solventar esto, se buscó una librería y existe una en quicklisp que se llama trivial-tco que se carga así (*ql:quickload :trivial-tco*)

Para probar el funcionamiento de esta librería, usaré Portacle (ya que no requiere tantas configuraciones para hacer funcional quicklisp). Se deben cargar y existir las siguientes funciones y bibliotecas

```
;carga las librerías de quicklisp
```

```
(ql:quickload :local-time)
```

```
(ql:quickload :cl-json)
```

```
(ql:quickload :trivial-tco)
```

;para determinar el color en un tiempo dado, se cambió el nombre para que no haya conflicto

;con la timer en Portacle. Uso el que tiene los valores fijos para probar.

```
timer_transition(data_time)
```

;para formatear correctamente las fechas

```
universal-to-datestring (utime &optional (timezone 0))
```

;para saber si se está en una transición o en un color fijo

```
traffic_ligth_status(data_time) ;para guardar en el archivo
```

```
txt_guardar_informe(tiempo_inicial ruta_archivo  
tiempo_max)
```

;para guardar desde un tiempo inicial hasta una cierta cantidad de segundos

```
(defun guardar_informe (i max_iter)
```

```
  (when (< i max_iter)
```

```
    (informe (+ (get-universal-time) i))
```

```
    (guardar_informe (1+ i) max_iter)))
```

Recordemos que la función clave es guardar_informe.


```
7209 DEBUG: estado = EN-ROJO
7210 DEBUG: estado = EN-ROJO
7211 DEBUG: estado = EN-ROJO
7212 DEBUG: estado = EN-ROJO
7213 DEBUG: estado = EN-ROJO
7214 DEBUG: estado = EN-ROJO
7215 DEBUG: estado = EN-ROJO
7216 DEBUG: estado = EN-ROJO
7217 DEBUG: estado = EN-ROJO
7218 DEBUG: estado = EN-ROJO
7219 DEBUG: estado = EN-ROJO
7220 DEBUG: estado = EN-ROJO
7221
7222 CL-USER> []

*slime-repl sbcl* 7222 Bot
587 ;ruta_archivo: La ruta del archivo. Se coloca una ruta por defecto
588 ;tiempo_max: La cantidad de segundos desde el tiempo_inicial en el cu
589
590 ;Salida: Muestra un mensaje con el estado en que se encuentra el semá
591 ;un archivo de texto la fecha y hora de la transición, así como la tr
592 ;=====
593
594
595 (defun guardar_informe (i max_iter)
596   (when (< i max_iter)
597     (informe (+ (get-universal-time) i))
598     (guardar_informe (1+ i) max_iter)))
599
600 (guardar_informe 0 3600)
601
602
603 ;=====
604 ;FINALIZA Extensión 2: Persistencia de Datos
605 ;=====

? /c/U/e/O/D/L/Paradigma/core.lisp 600 Bot 24C
=> NIL
```

La imagen anterior muestra cómo en Portacle si se ejecuta correctamente la función. Así que es muy recomendable usar Portacle en caso que se tenga problemas para cargar las bibliotecas con otras herramientas.

Elixir

Consignas

1. En Common Lisp, para la función `transicion` se suele usar un `cond` o `case`. En Elixir se utiliza *Pattern Matching* directo en los argumentos de la función (múltiples cláusulas). Muestren su código y expliquen por qué este enfoque es más declarativo.
2. Elixir utiliza el Modelo de Actores. Expliquen conceptualmente cómo se diseñaría un sistema donde cada semáforo de una esquina real sea un proceso independiente que se comunica enviando mensajes a los otros.

1. Uso de Pattern Matching en la función `transicion` y por qué es un enfoque más declarativo

Veamos nuestro código en lisp para la función `transición`
(defun transicion (color-actual cambiar-a)

```
(cond ((and (eq color-actual 'en-rojo) (eq cambiar-a 'amarillo))
      (list 'en-rojo "cambiar-a-amarillo"))
      ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'verde))
      (list 'en-amarillo "cambiar-a-verde"))
      ((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo))
      (list 'en-verde "cambiar-a-rojo"))
      (t
      (list color-actual "accion-por-defecto"))))
```

Notamos que para controlar que resultado retornará, tenemos tres condiciones señaladas en rojo, azul y naranja. Aunque estamos ante una programación funcional, este enfoque sigue siendo imperativo: debe decirse cómo controlar las condiciones y revisar si se cumplen o no. Si no se cumple, se revisa la siguiente hasta determinar cuál es la correcta.

En Elixir, se utiliza un concepto denominado coincidencia de patrones. Para entender esta coincidencia de patrones, usaremos un ejemplo señalado en el siguiente enlace:
https://elixirschool.com/es/lessons/basics/pattern_matching

```

iex> greeting = "Hello"
"Hello"
iex> greet = fn
...>   (^greeting, name) -> "Hi #{name}"
...>   (greeting, name) -> "#{greeting}, #{name}"
...> end
#Function<12.54118792/2 in :erl_eval.expr/5>
iex> greet.("Hello", "Sean")
"Hi Sean"
iex> greet.("Mornin'", "Sean")
"Mornin', Sean"

```

Imagen E1. Ejemplo de coincidencia de patrones

En este ejemplo, *greeting*="Hello" es una variable que guarda el string "Hello". Lo siguiente es la declaración de una función. Notamos que la función se llama *greet* y el bloque de la función comienza con *fn* y termina con *end*. Las siguientes líneas

```
(^greeting, name) -> "Hi #{name}"
```

```
(greeting, name) -> "#{greeting}, #{name}"
```

Indican el cuerpo de la función y allí podemos ver esta idea de patrones. La función sólo retornará un resultado la pregunta es ¿cuál?. El si nos centramos en ambas líneas notamos que del lado izquierdo de la flecha, la única diferencia es la aparición del operador ^. Este operador denominado operador pin, lo que hace es comparar si el parámetro *greeting* coincide con el contenido de la variable *greeting* retornará la línea "Hi #{name}". Pero si el contenido del parámetro *greeting* (el parámetro real de la función) no coincide con la variable *greeting* (la variable externa a la función), retornará la línea (greeting, name) -> "#{greeting}, #{name}"

Lo que hace Elixir es comparar qué patrón coincide y ejecuta la línea pertinente. Si se observa detenidamente, no hay operadores como cond, and, eq o similares. Esto hace que sea más declarativo la forma de escribir la función en Elixir: decimos qué debe ocurrir, no el programa debe verificarlo. Es como indicarle "estos son los casos válidos, selecciona el primero que encaje".

Podemos ver ahora el equivalente en Elixir para la función transicion

```
defmodule Transicion do
```

```
def transicion(:en-rojo, :amarillo) do
  {:en-rojo, "cambiar-a-amarillo"} end
```

```
def transicion(:en-amarillo, :verde) do
  {:en-amarillo, "cambiar-a-verde"} end
```

```
def transicion(:en_verde, :rojo) do
  {:en-verde, "cambiar-a-rojo"} end
```

```
def transicion(color-actual, cambiar-a) do
  {color-actual, "accion-por-defecto"} end
end
```

Es fácil ver que creamos un módulo llamado transición donde cada bloque es una función. En Elixir, los dos puntos es el equivalente al ‘ en Lisp. Los dos puntos sirven para enviar los átomos. Este bloque puede invocarse así

Transicion.transicion(:en_rojo, :amarillo)

Y al igual que el ejemplo anterior, la primera coincidencia que se cumpla será aquella que se ejecute. En nuestro caso, se ejecutará la función que retorne la tupla `{:en-rojo, "cambiar-a-amarillo"}`.

Nuevamente notamos que no hay operadores como `and`, `eq`, `cond` y demás. Se entiende mejor qué debe hacerse, sin decir expresamente cómo.

Una ventaja notable es que si necesitamos un patrón más, es fácil agregarlo sin toda la complejidad de colocar correctamente los operadores de comparación en lisp. Y también es más fácil ver cuál patrón debe ir primero: colocamos primero el patrón más general, y vamos bajando en generalidad para las diferentes opciones.

2. Diseño conceptual de un sistema de semáforos utilizando el Modelo de Actores de Elixir

Para empezar con la explicación conceptual, voy a basarme en algunas propiedades de Elixir denominadas Concurrencia OTP, Supervisores OTP y Distribución OTP. La idea es que cada semáforo sea considerado una máquina de estados finitos similar a esta

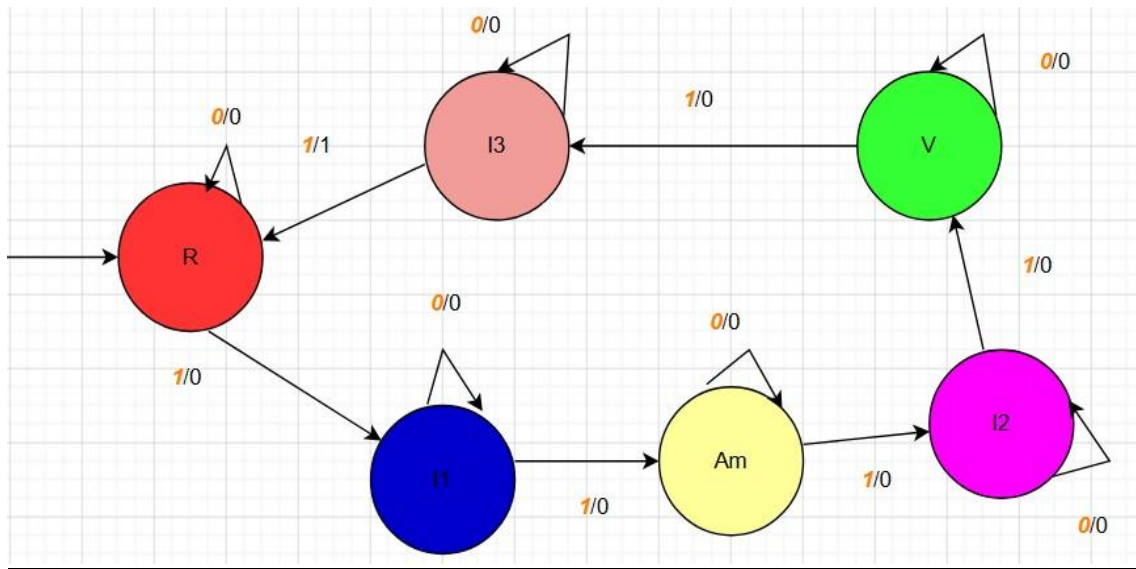


Imagen E2. Diagrama de la MEF para los semáforos.

Empezamos pensando solamente en dos semáforos que deben comunicarse entre sí. Cada semáforo será considerado un nodo. Podemos esquematizar los semáforos en un cruce de la siguiente manera

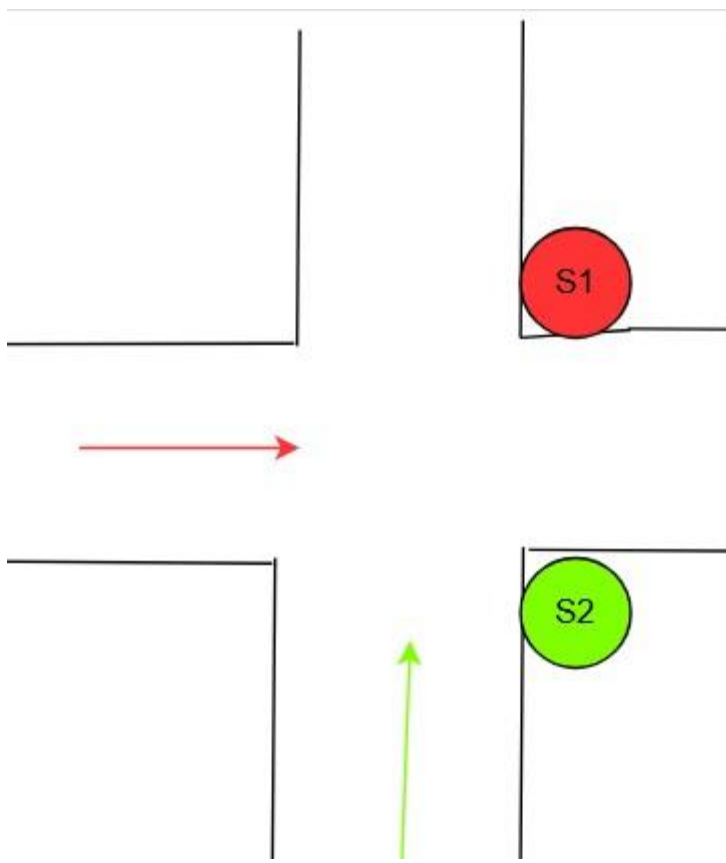


Imagen E3. Esquema de los semáforos

Podemos tener un proceso supervisor que se encargue, justamente, de supervisar a los semáforos en cada esquina. La idea de los supervisores es la siguiente: Los supervisores son procesos especializados con un propósito: monitorear otros procesos. Estos supervisores nos permiten crear aplicaciones tolerantes a fallos que automáticamente restauren procesos hijos en caso de falla.

Además, estos nodos pueden estar distribuidos: Podemos ejecutar aplicaciones Elixir en un conjunto de nodos distribuidos diferentes ya sea en un único *host* o en múltiples *hosts*. Elixir nos permite comunicarnos a través de estos nodos usando algunos mecanismos los cuales están fuera del objetivo de esta lección.

Entonces, teniendo en cuenta esta idea, podemos pensar en una entidad supervisora centralizada que se encargue de supervisar las comunicaciones entre los nodos. En esta idea, la entidad centralizada le dirá a los semáforos quien entrará en un ciclo rojo->amarillo->verde->rojo, mientras el otro se mantiene en rojo.

Esto podríamos esquematizarlo así.

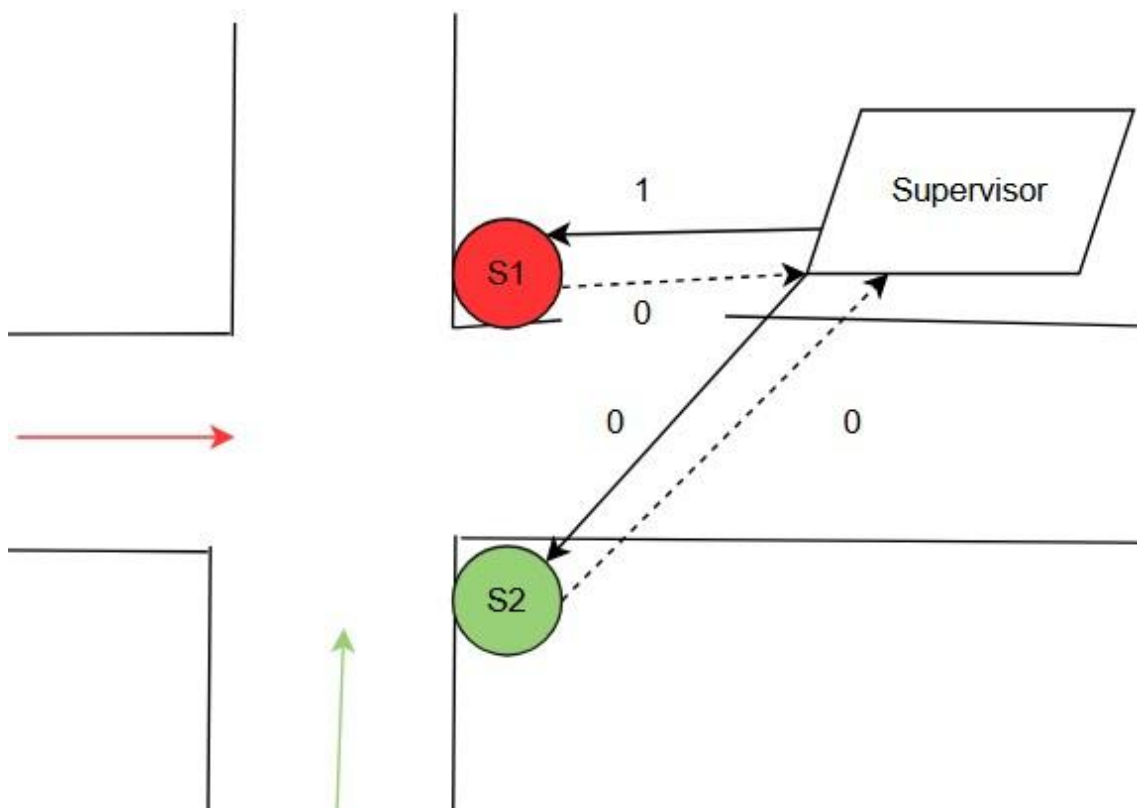


Imagen E4. Esquema de los semáforos con el supervisor.

Ambos semáforos estarán inicialmente en el estado R, que indica el estado en rojo. El supervisor puede tener una variable booleana que servirá para indicarle al supervisor a cuál semáforo debe supervisar, mientras el otro se mantiene en el estado R.

En el tiempo t_0 , cuando arranca todo, digamos que esa variable booleana está en True. Esto nos dice que el supervisor debe mandarle un 1 al semáforo S1 y un 0 al semáforo S2. Entonces, el

semáforo S1 pasará al estado I1 y el semáforo S2 permanecerá en el estado R. Ambos semáforos emiten un 0. Pero, como la variable booleana está en True, el supervisor estará al tanto sólo del mensaje emitido por S1. El 0 enviado S2 servirá para que el supervisor entienda que recibió su mensaje, pero no se usará por ahora para controlar el cambio de estados. Toda esta acción podría llevarse a cabo dentro de una función, a la cual podemos denominar *iniciar_ciclo*.

A la par, el Supervisor puede tener una lista de valores enteros. Supongamos que esa lista es la siguiente [3, 6, 3, 90, 3]. Estos representan valores para los contadores que se usarán en la comunicación entre S1 y el Supervisor. Cuando pasemos al tiempo t1, el temporizador arrancará en 3 y en cada tick del temporizador se producirá una comunicación como se muestra en la siguiente imagen

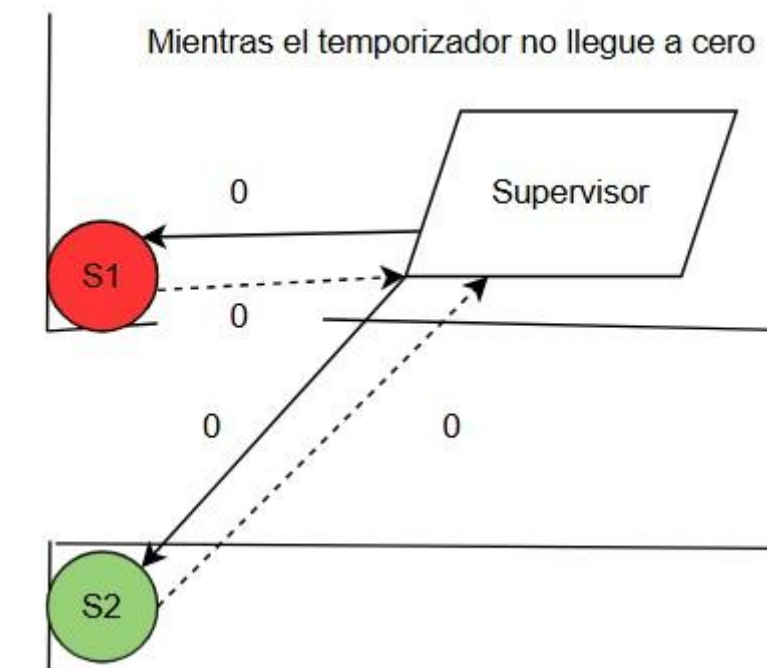


Imagen E5. Comunicación mientras el temporizador sea mayor a cero.

Mientras el temporizador sea mayor a cero, se seguirá enviando ceros a ambos semáforos, pero Supervisor sólo atiende a la respuesta enviada por S1 (mientras la variable booleana sea True).

Como S2 está en el estado R, al recibir un 0, emite un 0. Como S1 está en el estado I1, al recibir un 0, emite un 0.

Cuando el temporizador expire, se realiza esta comunicación

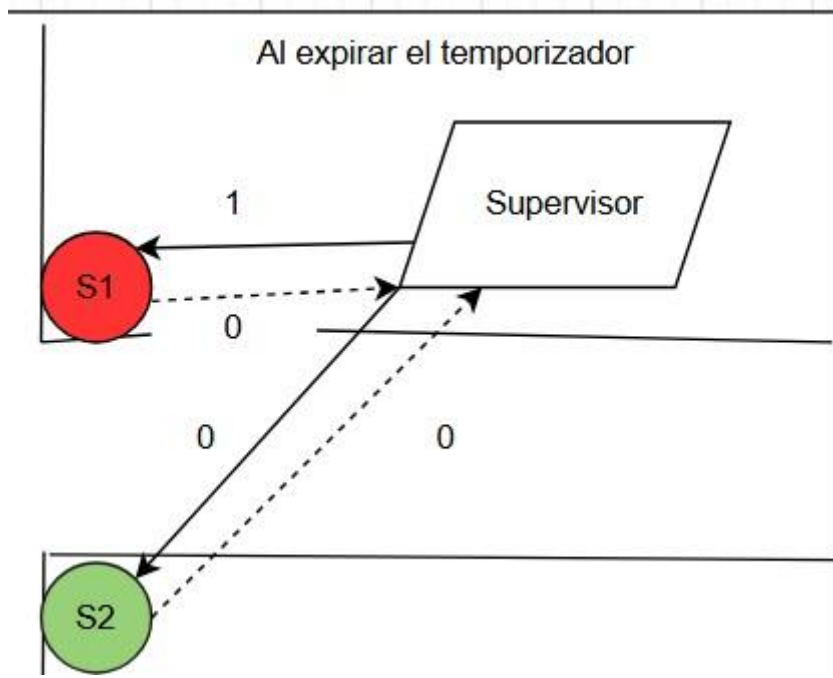


Imagen E5. Comunicación cuando el temporizador expira.

No hay cambios para S2, pero si S1 está en el estado I1 y recibe un 1, emite un cero y cambia al estado Am. Ahora, el temporizador se carga con el valor 6 (recordar la lista de tiempo [3, 6, 3, 90, 3], ya se usó el primer 3 para mantenerse en el estado I1, ahora se carga el tiempo 6 en el temporizador para mantenerse en el estado Am). Y nuevamente, se repite el ciclo y la comunicación es como en la imagen E4 mientras no expire el temporizador. Al expirar el temporizador, la comunicación es como en la imagen E5. Una vez más, S2 no cambia, pero S1 quien estaba en el estado Am pasa al estado I2 y emite un 0. Esto hace que el temporizador se cargue con el siguiente valor de la lista ([3, 6, 3, 90, 3], el 3 señalado en rojo aquí).

Podemos notar que esto se repetirá para los valores [3,6, 3, 90,3], [3,6, 3, 90,3], [3,6, 3, 90,3] sufriendo las transiciones I2->V->I3. Cuando esté en el estado I3 y expiren los últimos 3 segundos en el temporizador, el semáforo S1 pasará del estado I3 al estado R, pero esta vez emitirá un 1. Cuando el Supervisor reciba ese 1, cambiará la variable booleana de True a False y volverá a la cabeza de la lista [3, 6, 3, 90, 3], y el semáforo S1 transiciona al estado R. Esto hará que finalice un ciclo del semáforo. Podemos introducir todo esto en una función llamada *realizar_ciclo*.

Cuando finaliza la función anterior, volvemos a ejecutar la función *iniciar_ciclo*. Pero como ahora la variable booleana es False, la nueva comunicación es la siguiente

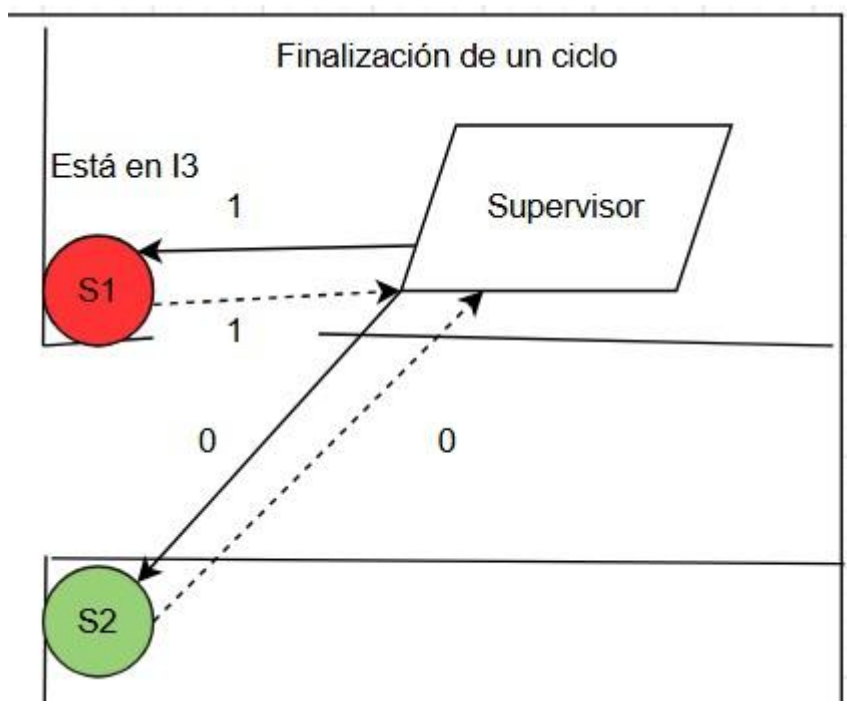


Imagen E6. Finalización de un ciclo

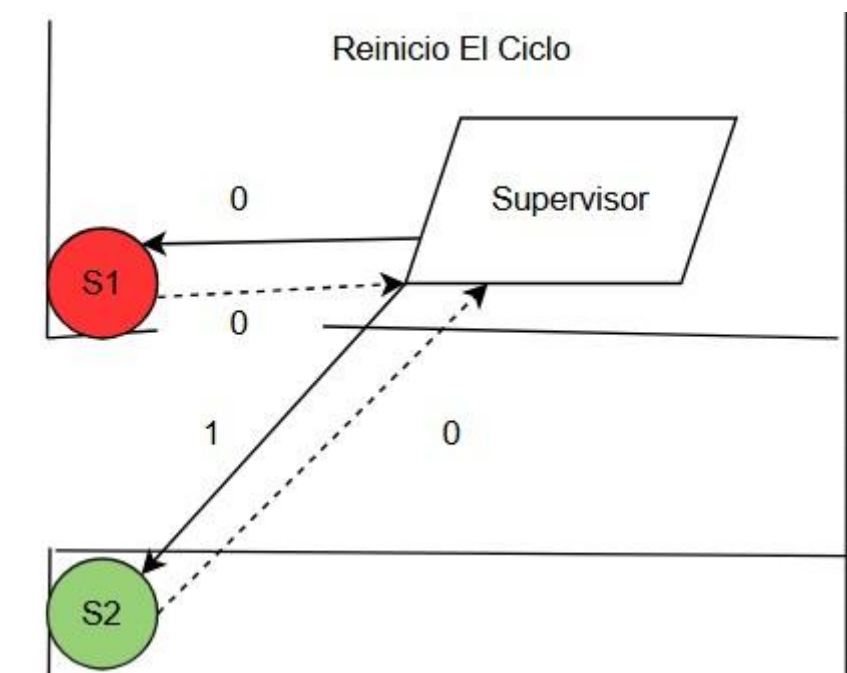


Imagen E7. Inicio de un nuevo ciclo.

A continuación, el ciclo se repite, pero como la variable booleana está en False, el Supervisor sabe que debe empezar a hacer caso a lo que le emite el semáforo S2 y le va enviando 0 al semáforo S1 para que se mantenga en el estado R. Los temporizadores pasarán nuevamente por los tiempos de la lista de tiempos, hasta que al expirar el último temporizador y emitir el supervisor el último 1 al semáforo S2 éste le retorne un 1. Para empezar el ciclo nuevamente con el semáforo S1.

Por simplicidad, se supuso que el tiempo que toma transicionar de un estado a otro es despreciable, así como los demás tiempos involucrados en la transmisión de los mensajes (por ejemplo, el tiempo de propagación).

Conclusión sobre la experiencia de estudio y utilización de Elixir

El desarrollo de este Trabajo Práctico Integrador permitió analizar e investigar elixir y resultó ser muy enriquecedora, ya que brindo conocer un lenguaje funcional moderno con características diferentes a las de common Lisp. Durante el estudio se observó que conceptos como el *pattern matching*, la inmutabilidad de los datos y el modelo de actores ofrecen formas más directas y expresivas de resolver determinados problemas, especialmente aquellos relacionados con la concurrencia y la comunicación entre procesos.

Si bien al principio fue necesario familiarizarse con una sintaxis distinta y con conceptos propios del ecosistema OTP, el análisis permitió comprender cómo Elixir facilita el desarrollo de aplicaciones robustas, escalables y tolerantes a fallos. Además, la comparación con Common Lisp ayudó a identificar ventajas y limitaciones de cada enfoque, fortaleciendo nuestra comprensión de los paradigmas funcionales.

En términos generales, la experiencia fue positiva porque amplió nuestra visión sobre las diferentes formas de abordar un mismo problema computacional, adquirir conocimientos sobre tecnologías actuales utilizadas en sistemas distribuidos y de alta concurrencia, enriqueciendo nuestra formación y comprender cómo diferentes tecnologías pueden abordar una misma problemática desde enfoques distintos.

Bitácora de 5

Error N.º 1: Envío accidental de listas en lugar de símbolos

Código involucrado

```
(transicion '(1 2) '(3 4))
```

Problema observado

La función intentaba procesar listas cuando el diseño del sistema esperaba únicamente símbolos.

El resultado obtenido era:

```
((1 2) "accion-por-defecto")
```

```

1 (defun transicion (color-actual cambiar-a)
2
3   (cond
4     ((and (eq color-actual 'en-rojo) (eq cambiar-a 'amarillo))
5      (list 'en-rojo "cambiar-a-amarillo"))
6
7     ((and (eq color-actual 'en-amarillo) (eq cambiar-a 'verde))
8      (list 'en-amarillo "cambiar-a-verde"))
9
10    ((and (eq color-actual 'en-verde) (eq cambiar-a 'rojo))
11     (list 'en-verde "cambiar-a-rojo"))
12
13    (t
14     (list color-actual "accion-por-defecto"))))
15 (transicion '(1 2) '(3 4))

```

```

TRANSICION
Break 5 [6]>
TRANSICION
Break 5 [6]>
((1 2) "accion-por-defecto")
Break 5 [6]>

```

Causa

No existía inicialmente una validación que impidiera el ingreso de listas como parámetros.

Solución

Se agregó la validación:

```

((or (listp color-actual)
      (listp cambiar-a))
  NIL)

```

```

1 (defun control-cambio-color (color-actual cambiar-a)
2
3   (cond
4
5     ((or (listp color-actual)
6          (listp cambiar-a))
7      NIL)
8
9     (t
10      (transicion color-actual cambiar-a))))
11 (control-cambio-color '(1 2) '(3 4))

```

```

NIL
Break 5 [6]>
CONTROL-CAMBIO-COLOR
Break 5 [6]>
NIL
Break 5 [6]>

```

De esta forma la función rechaza estructuras inválidas y evita resultados inconsistentes.

Error N.º 2: Confusión entre tiempo Unix y tiempo Universal

Código involucrado

(get-universal-time)

Problema observado

Los cálculos temporales generaban resultados incorrectos al utilizar directamente el tiempo devuelto por Lisp.

Causa

Common Lisp utiliza como referencia temporal el 1 de enero de 1900, mientras que Unix utiliza el 1 de enero de 1970.

Al utilizar directamente el valor retornado por:

(get-universal-time) los cálculos quedaban

desplazados en varios años.

Solución

Se creó la función:

(defun get-unix-time ())

(- (get-universal-time) 2208988800)) restando la diferencia de segundos entre ambas fechas de referencia.

```
[1]>  
3998624986  
[2]>  
1781636144  
[3]>
```

(Common Lisp devuelve tiempo desde 1900. Unix cuenta desde 1970. Hubo que restar 2208988800 segundos.)

Error N.º 3: Imposibilidad de consultar fechas específicas

Código involucrado

(timer (get-unix-time))

Problema observado

Inicialmente sólo era posible conocer el estado actual del semáforo.

No se podía consultar qué color tenía en una fecha y hora determinada.

Causa

La función trabajaba únicamente con el tiempo presente y no permitía analizar eventos históricos.

Solución

Se implementaron las funciones:

(parse-fecha "2025-06-13 10:30:00") y

(diferencia-tiempo "2025-06-13 10:30:00")

que permiten convertir fechas ingresadas por el usuario y calcular la diferencia respecto del tiempo actual.

Gracias a esto fue posible consultar el estado del semáforo para momentos específicos y mejorar las tareas de auditoría y análisis temporal.

```
39 (diferencia-tiempo "2025-06-13 10:30:00")  
40 (timer (diferencia-tiempo "2025-06-13 10:30:00"))  
  
DIFERENCIA-TIEMPO  
[14]>  
31826111  
[15]>  
EN-ROJO  
[16]>
```

En la imagen se observa la ejecución de la función diferencia-tiempo, la cual calcula la cantidad de segundos transcurridos entre una fecha específica ingresada por el usuario y el momento actual. Posteriormente, dicho valor se utiliza como entrada de la función timer, obteniéndose como resultado el estado del semáforo para esa fecha.

Error N.º 4: Temporizador dependiente de valores fijos

Código involucrado

```
1 (defun timer_transition(data_time)
2
3   (cond
4     ((< (mod data_time 216) 90) 'en-rojo)
5     ((and (>= (mod data_time 216) 90)
6           (<= (mod data_time 216) 95))
7       'en-amarillo)
8     ((and (>= (mod data_time 216) 96)
9           (<= (mod data_time 216) 215))
10        'en-verde)))
11 |
```

Problema observado

Cada vez que se modificaba la duración de algún color era necesario cambiar manualmente varios valores dentro de la función.

Causa

La lógica dependía de números fijos ("valores mágicos") calculados para un único escenario:

Rojo = 90 segundos

Amarillo = 6 segundos

Verde = 120 segundos

Total = 216 segundos

Si alguno de esos tiempos cambiaba, el temporizador dejaba de funcionar correctamente.

Solución

Se reemplazaron los valores fijos por parámetros: (defun timer_2 (tiempo-unix rojo amarillo verde) y se calculó dinámicamente la duración del ciclo mediante:

(+ rojo amarillo verde) permitiendo reutilizar la función con cualquier configuración.

```
CONTROL_TIMER
Break 2 [18]>
EN-VERDE
Break 2 [18]>
EN-VERDE
Break 2 [18]>
```

Error N.º 5: Falta de validación de datos de entrada

Código involucrado

(timer_2 tiempo-unix rojo amarillo verde)

Problema observado

La función aceptaba tiempos negativos o duraciones iguales a cero.

Por ejemplo:

```
1 (timer_2 -100 90 6 120)
2 (timer_2 1000 0 6 120)
```

```
EN-VERDE
Break 2 [18]>
EN-VERDE
Break 2 [18]>
EN-VERDE
Break 2 [18]>
```

Causa

No existían controles previos que verificaran la validez de los datos recibidos.

Solución

Se implementó una función de control:

(defun control_timer(tiempo-unix rojo amarillo verde)

(cond

((< tiempo_unix 0)

"El tiempo no puede ser menor a cero")

((<= rojo 0)

"El tiempo del semáforo en rojo no puede ser cero o menos")

(t

(timer_2 tiempo-unix rojo amarillo verde))))

De esta forma se evita ejecutar el temporizador con valores inválidos

```
12 (control_timer -100 90 6 120)
13 (control_timer 1000 0 6 120)
14 (control_timer 1000 90 6 120)
```

```
"El tiempo no puede ser menor a cero"
Break 2 [18]>
"El tiempo del semáforo en rojo no puede ser cero o menos"
Break 2 [18]>
EN-VERDE
Break 2 [18]>
```

Error N.º 6: Incompatibilidad de bibliotecas en CLISP

Código involucrado

```
(ql:quickload :local-time)
```

Problema observado

Durante la implementación de la Fase 2 fue necesario utilizar la biblioteca **Local-Time** para trabajar con fechas y horas en formato legible. Sin embargo, al intentar cargar la biblioteca en CLISP surgieron errores de compatibilidad y dependencia que impedían su correcto funcionamiento.

Causa

La biblioteca Local-Time presenta limitaciones de compatibilidad con algunas versiones de CLISP. Debido a ello, Quicklisp no lograba cargar correctamente todas las dependencias necesarias para ejecutar las funciones relacionadas con fechas y timestamps.

```
*** - READ en #<INPUT CONCATENATED-STREAM #<INPUT STRING-INPUT-STREAM> #<INPUT UNBUFFERED
FILE-STREAM CHARACTER @2>>: no
    existe ningún paquete con el nombre "QL"
```

Solución

Se investigaron alternativas y se decidió utilizar **Portacle**, una distribución de Common Lisp que ya incorpora Quicklisp y ofrece una mejor compatibilidad con bibliotecas modernas.

A partir de este cambio fue posible cargar correctamente la biblioteca:

```
(ql:quickload :local-time) y
```

utilizar funciones como:

```
(local-time:now)
```

```
(local-time:timestamp-difference ...)
```

permitiendo implementar las funcionalidades de auditoría temporal solicitadas en el trabajo práctico.


```

1 ; SLIME 2.24
2 CL-USER> (ql:quickload :local-time)
3 To load "local-time":
4   Load 1 ASDF system:
5     local-time
6 ; Loading "local-time"
7
8 (:LOCAL-TIME)
9 CL-USER> (local-time:now)
10 @2026-06-16T19:38:34.845204Z
11 CL-USER> (local-time:timestamp-difference
12   (local-time:now)
13   (local-time:now))
14 0
15 CL-USER>

```

Referencias

Definición De Tiempo Unix: <https://espanol.epochconverter.com/>

Definición de Universal Time en Lisp:

https://lispcookbook.github.io/clcookbook/dates_and_times.html

Prompt De ChatGPT: Cómo se obtiene en lisp esto: Tiempo Unix actual (entero)

CL-JSON Documentation

Referencia utilizada para la lectura de archivos JSON.

[GitHub Documentation](#)

Utilizada para la gestión del repositorio y control de versiones.

Local-Time Library

Biblioteca utilizada para el manejo de fechas y horarios.

Para las imágenes se utilizo la herramienta: [Diagrama - draw.io](https://draw.io)

Markdown: lenguaje de marcado utilizado para la elaboración de documentación técnica y del archivo HONOR.md del repositorio GitHub. <https://sourceforge.net/projects/portecle/>

<https://elixir-lang.org/learning.html> <https://elixir.hexdocs.pm/introduction.html>

<https://elixirschool.com/es/lessons/basics/basics>